

REDROB HACKATHON

Intelligent Candidate Recovery

A retrieval + scoring pipeline that recovers the strongest ML / search engineering candidates from a noisy 100K-profile pool

TEAM

Amit Sharma

Keshav Varshney



The Problem

Finding the right ML / search engineers inside a huge, messy candidate pool



Scale

~100,000 raw candidate profiles in JSONL — far too many to review manually within a hackathon window.



Noise & Irrelevance

Most profiles belong to unrelated roles (HR, sales, finance) that dilute retrieval and waste compute.



Honeypot Profiles

Fabricated or inflated resumes (expert skills with zero tenure, impossible timelines) can game naive rankers.



Time Budget

Needed an accurate, defensible top-100 shortlist with reasoning — computed in minutes, not hours, on CPU only.

Our Approach

Filter early, retrieve smart, score deep — only where it matters



1. Hard Filter

Rule-based role classification drops Tier-D (irrelevant roles) and honeypot profiles before any embedding — cutting the corpus 40–60% for free.



2. Hybrid Retrieval

Semantic search (Sentence-Transformers) + lexical search (BM25) run on the reduced corpus and their top-K results are unioned.



3. Weighted Scoring

Only the shortlist (~3-4K) gets full feature scoring: role fit, experience validity, skills, location, education, GitHub, availability.



4. Ranked Output

Top 100 candidates, sorted by composite score, each with an auto-generated natural-language reasoning string.

Pipeline Architecture

Two-phase design: heavy lifting offline, ranking online

PHASE A — OFFLINE (once)

- Stream & flatten 100K JSONL candidates
- Rule-based role tiering (A/B/C/D) + evidence check
- Honeypot detection (skill / timeline / tenure flags)
- Hard-filter Tier-D & honeypots before embedding
- Load MiniLM-L6-v2 once, encode remaining corpus
- Build BM25 index on the same reduced corpus
- Persist features (Feather), embeddings (.npy), BM25 (pkl)

PHASE B — ONLINE (per query)

- Load cached features / embeddings / BM25 (fast)
- Reuse in-memory model — embed the JD once
- Cosine-similarity rank -> Embed Top-K (3,000)
- BM25 lexical rank -> BM25 Top-K (1,000)
- Union both shortlists as the candidate pool
- Full weighted scoring only on the shortlist
- Sort, take Top 100, generate reasoning, write CSV

Hybrid Retrieval: Semantic + Lexical

Neither signal alone is reliable — union them to catch true positives either would miss



Semantic Search

- Model: sentence-transformers/all-MiniLM-L6-v2
- Candidate text = headline + summary + titles + career descriptions (truncated to 128 words)
- Embeddings L2-normalized -> cosine similarity via dot product
- Model loaded once and reused across offline + online phases
- Captures conceptual fit: "recommender systems" ~ "ranking engineer"



Lexical Search (BM25)

- Okapi BM25 over tokenized candidate text
- Built only on the already-filtered, embeddable subset — not the full 100K
- Catches exact keyword matches embeddings can blur: "NDCG", "FAISS", "Pinecone"
- JD tokenized once, scored against the cached index per run
- Embed Top-3000 U BM25 Top-1000 -> final shortlist

Composite Scoring Model

Six weighted signals combined, then adjusted for availability and role-tier trust



40%

Role Fit

Title tiering (A/B/C/D) blended with recent-job career identity



25%

Experience

Concept-hit density, role-months recency, years-of-exp curve, production bonus



15%

Skills

Taxonomy relevance x proficiency/assessment trust, saturating curve



10%

Location

City tier + work-mode preference logistics score



5%

Education

Field relevance x institution tier



5%

GitHub

Small bonus for strong public activity score

$final_score = (weighted\ sum) \times availability_multiplier$ (×0.02 if Tier-D)

Trust & Safety Filters

Cheap, rule-based checks that keep fabricated or irrelevant profiles out of the shortlist



Honeypot Detection

- Skill flag: 3+ "advanced/expert" skills with 0-month duration
- Timeline flag: claimed YOE exceeds earliest-job span by 3+ years
- Tenure flag: any single job's duration exceeds total claimed YOE
- Flagged profiles are excluded before embedding — saving compute and preventing gamed rankings



Role-Tier Gate

- Tier A: direct ML/search/ranking/NLP titles — full weight
- Tier B: adjacent eng/data titles + ML evidence in text
- Tier C: general dev/full-stack — partial credit only
- Tier D: HR, sales, finance, etc. — dropped pre-embedding, and any survivor gets a 0.02x score multiplier

Performance Engineering

CPU-only runtime cut from 15+ minutes to 5-7 minutes

15+

min — before



5-7

min — after

Filtering before embedding is the single biggest win:

skipping irrelevant profiles means the expensive semantic-encoding step only ever touches candidates that could plausibly rank.

Key Optimizations

- Hard role-D + honeypot filter before embedding (~40-60% corpus)
- Embedding model loaded once, reused online
- Vectorized / list-comprehension scoring, no row-wise apply in hot paths
- BM25 built only on the filtered subset, not full 100K
- Larger batch size (512) for CPU throughput
- Text capped at 128 words — matches model's max_seq_length
- Feather I/O instead of pickle (3-5x faster df serialization)

Tech Stack

Lightweight, CPU-friendly, notebook-native

Python 3

Core language

Google Colab

Execution environment

sentence-transformers

MiniLM-L6-v2 embeddings

rank-bm25

Okapi BM25 lexical search

pandas / NumPy

Data wrangling & vectorized scoring

scikit-learn

Utility ML helpers

PyArrow (Feather)

Fast DataFrame serialization

python-docx

Parsing JD / spec documents

tqdm

Progress visibility on long loops

Output & Validation

A clean, ranked, explainable submission — validated before delivery

submission.csv

candidate_id	rank	score	reasoning
c_04831	1	0.912	ML Engineer, 7.2 yrs...
c_19204	2	0.897	Research Engineer...
c_00113	3	0.884	AI Engineer, embeddings...

- Rank 1-100, sorted by composite score
- Auto-generated reasoning per candidate (skills, availability, notice period, activity recency)
- Schema asserted before write: exact 4 columns, exact 100 rows, ranks 1-100



Validation & Delivery

- validate_submission.py run automatically against the output
- Return code, stdout, and stderr all logged for transparency
- Final CSV copied back to the shared Drive folder
- Local download triggered for the notebook runner
- Diagnostics printed: honeypots and Tier-D count in the final Top 100 (sanity check, should be 0)

Thank You

Intelligent Candidate Recovery — built for speed, precision, and explainability

Amit Sharma • Keshav Varshney

Redrob Hackathon Submission

